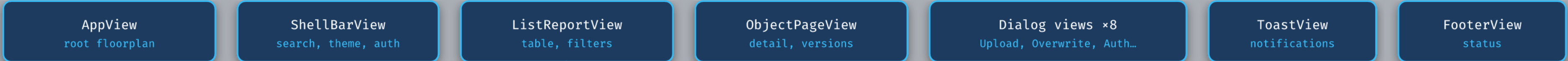


Personal Library – Frontend Component Interaction

Hand-rolled runtime: Component views • observable Stores • typed event bus • swappable adapters – no framework

Component / Data Flow

VIEW LAYER – Component subclasses (core/component.ts)



INTENT CALLS – plain method invocation
applyFilters() • openDocument() • openUpload() • logout()

SUBSCRIBE / NOTIFY – the only path back up
store.subscribe() ⇒ this.requestRender()
coalesced into one microtask; setState() short-circuits when nothing actually changed

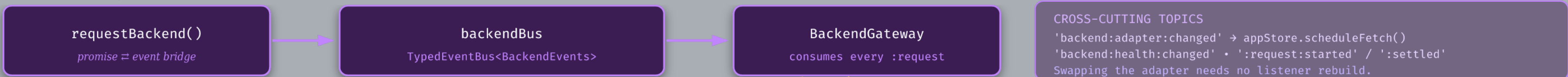
STATE LAYER – Store<S> subclasses (core/store.ts)



requestBackend(op, payload)

resolve / reject → setState()

MESSAGING LAYER – services/backend (core/eventBus.ts)



CROSS-CUTTING TOPICS
'backend:adapter:changed' → appStore.scheduleFetch()
'backend:health:changed' • ':request:started' / ':settled'
Swapping the adapter needs no listener rebuild.

publish ':request' → handler → publish ':success' / ':failure' (correlated by requestId)

invoke(op, resolveAdapter(), request)
the dashed path is the Mock adapter – same contract, no network

ADAPTER LAYER – exactly one active at a time



fetch() through the nginx ingress
SessionService renews on the dashed path: POST /auth/refresh

BACKEND TIER



WHY AN EVENT BUS AT ALL

A direct store → adapter call would work, but every request would then be invisible. Routing through topics makes the whole conversation observable, lets the gateway time and normalise errors in one place, and lets the adapter be swapped at runtime without rebuilding a single subscription.

BackendGateway.execute() • services/backend/BackendGateway.ts

PROMISES OVER AN EVENT BUS

requestBackend() generates a requestId, subscribes to the matching :success and :failure topics, publishes the request, then resolves or rejects and disposes both listeners. Callers write plain await; the message still crosses the bus. Envelopes whose requestId does not match are ignored.

requestBackend() • services/backend/BackendGateway.ts

LIFECYCLE AND LEAK SAFETY

Every subscription is registered through Component.track(), so destroy() disposes store listeners, bus listeners and timers together. render() replaces innerHTML wholesale, then restores focus and scroll from a snapshot – which is what keeps typing in a filter field from losing the caret on a background refresh.

core/component.ts • track() / render() / destroy()